

Performance Evaluation of the Backpressure Collection Protocol (BCP) over Static Wireless Networks

Chin-Wei Huang

Ming Hsieh *Department of Electrical and Computer Engineering
University of Southern California*

Los Angeles, California

Abstract

This paper presents the implementation and evaluation of the Backpressure Collection Protocol (BCP) over a static wireless network using the PyWiSim simulator. Motivated by the need for efficient dynamic routing without explicit path maintenance, the study contrasts BCP against a naive Random Walk baseline to highlight backpressure’s routing efficiency. Furthermore, it investigates the critical impact of queue management by comparing Last-In-First-Out (LIFO) and First-In-First-Out (FIFO) scheduling mechanisms. By analyzing queue backlog states, we empirically justify why LIFO scheduling achieves significantly lower delay for delivered packets in backpressure environments. An extensive numerical evaluation of the network’s stability region reveals that BCP sustains a maximum throughput capacity of approximately 0.85 packets per second, drastically outperforming the Random Walk baseline (0.18 packets per second). The simulation code and experimental data are provided in a public repository for full reproducibility.

Index Terms— Backpressure routing, BCP, wireless networks, LIFO scheduling, PyWiSim, Stability Region.

1 Introduction

Routing in wireless sensor and ad-hoc networks faces unique challenges due to dynamic link conditions, interference, and fluctuating topologies. Traditional routing protocols establish explicit paths, which can become brittle and incur significant overhead when links fail. To address this, the Backpressure Collection Protocol (BCP) was introduced as a dynamic routing algorithm that makes per-packet routing and scheduling decisions based on local queue differentials and link state information [1]. By forwarding packets to neighboring nodes that offer the highest positive queue gradient, BCP inherently mitigates congestion and dynamically routes around poor link conditions without requiring end-to-end path discovery.

The motivation of this research is to explicitly evaluate the practical performance of BCP in a static wireless mesh network, focusing on the interplay between the protocol’s routing gradients and queue scheduling disciplines. Specifically, we aim to demonstrate why tradi-

tional First-In-First-Out (FIFO) queue management performs poorly in backpressure routing and why Last-In-First-Out (LIFO) is strictly necessary to minimize delivery delay.

Conducted as a final research project for EE 597 at the University of Southern California [2], this work implements BCP and a Random Walk baseline over the PyWiSim simulation framework. We define a rigorous system model and evaluate both stability limits and delay distributions, providing a comprehensive empirical analysis of backpressure dynamics.

2 System Model

To evaluate the protocol, we define a time-slotted wireless network model characterized by specific queuing and arrival dynamics.

2.1 Network and Channel Model

The system is modeled as a static wireless mesh network where nodes are distributed in a geometric coordinate plane. Time is slotted, with a fixed time slot length of 1 second. The wireless medium supports a fixed link rate of exactly 1 packet per time slot. The channel exhibits path loss, resulting in a distance-dependent packet delivery probability that informs Expected Transmission Count (ETX) estimations.

2.2 Queuing Dynamics

Each node maintains an internal data queue of infinite capacity to buffer incoming packets. During each time slot, a node can transmit at most one packet to a neighboring node, provided the scheduling algorithm permits it.

2.3 Arrival Model

We assume a deterministic arrival model for exogenous traffic. Packets are generated at a source node according to a specified arrival rate λ (measured in packets/second). This is implemented by injecting packets uniformly and deterministically across the available time slots.¹

¹For example, an arrival rate of $\lambda = 0.2$ means exactly one exogenous packet arrives every 5 time slots. This ensures a strict

3 Protocol Description

In our PyWiSim implementation, each node executes the protocol logic at every time slot. The wireless medium utilizes Carrier Sense Multiple Access (CSMA) for medium access control, which is natively implemented within the PyWiSim framework to handle collision avoidance and manage concurrent transmissions. Nodes periodically exchange their current queue sizes with neighbors via local broadcasts (simulated as accessible local state).

3.1 BCP Weight Calculation

The scheduling decision in BCP is governed by a weight function that incorporates both the queue differential and a link penalty (ETX). For any node i considering a transmission to a neighbor j , the link weight W_{ij} is calculated as:

$$W_{ij} = (Q_i - Q_j) - V \cdot \text{ETX}_{ij} \quad (1)$$

where Q_i and Q_j represent the backlog sizes at nodes i and j , respectively. The control parameter $V \geq 0$ dictates the strictness of the penalty for poor links; higher V favors links with fewer expected retransmissions, while $V = 0$ strictly follows the queue gradient. A node will transmit its packet to the neighbor j^* that maximizes W_{ij} , provided that $W_{ij^*} > 0$. If no neighbor yields a strictly positive weight, the node retains its packet for that time slot.

3.2 Random Walk Baseline

To quantify the efficiency of BCP's gradient-based routing, we implemented a Random Walk protocol as a naive baseline. In this protocol, queue gradients and ETX penalties are completely ignored. Instead, if a node has a non-empty queue, it randomly selects one of its physical neighbors and attempts to forward the packet.

4 Evaluation Setup

The protocols were evaluated using the PyWiSim framework over the 7-node topology depicted in Fig. 1. Node A acts as the singular traffic source, and Node G acts as the singular data sink.

Two distinct simulation tests were conducted to evaluate different performance metrics:

- **Test 1 (Queue State Analysis):** A short-duration simulation consisting of the first 20 seconds with 20 exogenous packets (arrival rate $\lambda = 1.0$) and 5 more seconds with no exogenous packets. This test aims to isolate the internal queue states and trace individual packet delays to observe the exact effect of LIFO versus FIFO scheduling.

deterministic arrival pattern rather than a memoryless Poisson process.

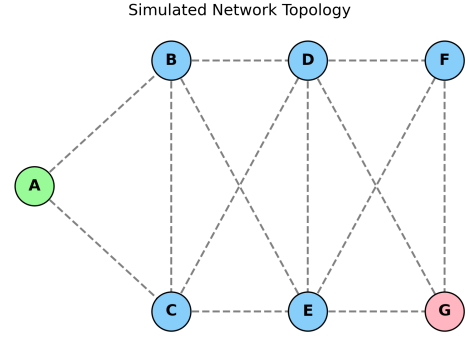


Figure 1: The 7-node static mesh network topology evaluated in PyWiSim. Node A serves as the traffic source, and Node G acts as the data sink.

- **Test 2 (Stability Region Analysis):** A long-duration simulation generating 10,080 packets per run. The arrival rate λ was swept from 0.2 to 2.0 packets/second to empirically determine the maximum sustainable throughput (the stability region) and observe the corresponding scaling of packet delay.

5 Results and Analysis

5.1 Queue State and LIFO Justification

Traditional queue management relies on FIFO scheduling. However, backpressure routing fundamentally requires a "gradient" of backlogged packets across the network to push data toward the sink. If FIFO is used, newly arrived packets must wait behind the existing gradient, resulting in severe delay.

Table 1: End-of-Simulation Queue States (20 pkts)

Node	Size	FIFO Content	LIFO Content
A	3	Pkts 17–19	Pkts 0, 3, 8
B	2	Pkts 14, 16	Pkts 1, 6
C	2	Pkts 13, 15	Pkts 2, 7
D	1	Pkt 12	Pkt 4
E	1	Pkt 11	Pkt 5
F	1	Pkt 0	Pkt 9
G	0	Older Arrived	Newer Arrived

To demonstrate this, we analyzed the final queue states of Test 1 (Table 1). The backlog sizes across all intermediate nodes are strictly identical for both scheduling disciplines, perfectly illustrating the progressive routing gradient ($A = 3 \rightarrow B, C = 2 \rightarrow D, E, F = 1 \rightarrow G = 0$). However, the *content* of these queues drastically differs:

- **Under FIFO:** Older packets successfully reach the sink, but newer packets (e.g., Pkts 11–19) remain trapped in the queues of intermediate nodes (A–E) to form the necessary gradient.
- **Under LIFO:** Older packets (e.g., Pkts 0–9) are retained across all intermediate nodes (A–F) to establish a static gradient. This allows newer packets to “surf” over the established gradient and rapidly reach the sink.

This queue state analysis empirically proves that LIFO scheduling is essential for backpressure networks, as it minimizes delivery delay by utilizing older packets merely as the structural gradient.

5.2 Throughput and Stability Region

Fig. 2 details the results of Test 2. We observe that BCP (both LIFO and FIFO) scales linearly with the arrival rate until it reaches a stability limit of approximately **0.85 packets/second**. This capacity ceiling is a direct consequence of the specific network topology, the shared wireless nature of the medium, and the CSMA mechanism, which collectively restrict the number of concurrent non-interfering routes and limit their effective transmission rates. Beyond this threshold, the network is saturated. In stark contrast, the naive Random Walk baseline saturates almost immediately, capping at a maximum throughput of roughly **0.18 packets/second** due to highly inefficient, undirected ping-ponging of packets between nodes.

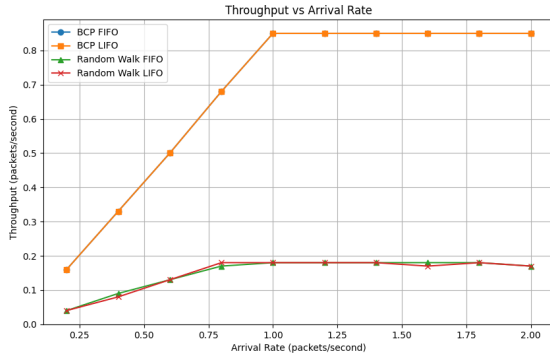


Figure 2: Network throughput versus packet arrival rate. BCP sustains a capacity of 0.85 pkt/s, far outperforming the Random Walk baseline.

5.3 Packet Delay Scaling

The choice of queue scheduling profoundly impacts the delay under varying loads, both below and above the network’s stability limit.

Fig. 3 highlights the delay performance purely within the stable regime. Even when the network is uncongested,

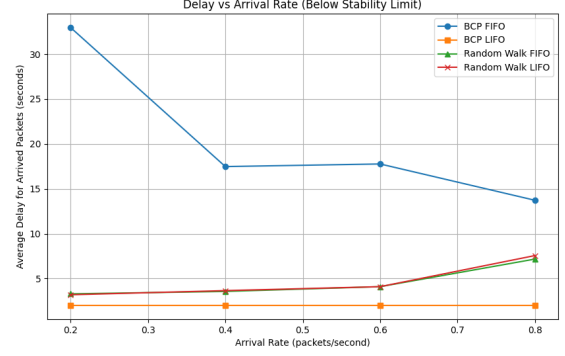


Figure 3: Average delay for successfully delivered packets below the stability limit (≤ 0.8 pkt/s).

BCP-LIFO exhibits a significant advantage over BCP-FIFO, maintaining a remarkably constant and low delivery delay of **2.01 seconds**. In contrast, BCP-FIFO incurs a significantly higher delay (ranging from 13 to 33 seconds) because newly arrived packets must wait for the structural queue gradients to flush through the network.

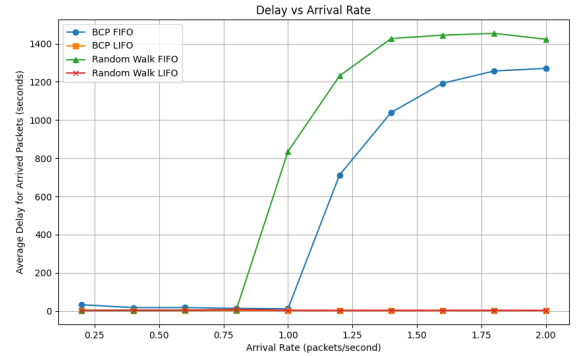


Figure 4: Average delay across all arrival rates, illustrating the unbounded queue growth for FIFO above capacity.

This advantage becomes exponential once the arrival rate pushes the network past its capacity boundary (0.85 pkt/s), as shown in Fig. 4. While BCP-LIFO continues to guarantee a 2.01-second delivery delay by allowing fresh packets to immediately bypass the saturated queues, BCP-FIFO sees its delivery delay skyrocket to over 1,200 seconds. Because FIFO forces packets to wait behind an infinitely growing backlog, its delay scales unboundedly above the stability limit, effectively trapping all newly generated traffic.

6 Conclusion

This project successfully implemented and evaluated the Backpressure Collection Protocol against a Random Walk

baseline in a simulated wireless environment. The results clearly outline BCP’s superior stability region (0.85 pkt/s vs 0.18 pkt/s). Through queue backlog analysis, we provided concrete empirical justification for LIFO scheduling in backpressure networks.

While BCP-LIFO guarantees a remarkably low delivery delay by maintaining routing gradients using older packets, a notable limitation of this study is the assumption of a static topology and a single traffic flow. Future work could explore multi-commodity flows and the impact of dynamic node mobility on gradient stability.

Code Repository

The complete PyWiSim simulation code, protocol implementations, and raw experimental data are publicly accessible at: <https://github.com/ChinWeiHuang/EE597-BCP-PyWiSim>

References

- [1] S. Moeller, A. Sridharan, B. Krishnamachari, and O. Gnawali, “Routing without routes: the backpressure collection protocol,” in *Proceedings of the 9th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*, 2010, pp. 279–290.
- [2] B. Krishnamachari, “EE 597: Wireless Networks,” Course Materials, University of Southern California, 2026.